

Commute Intelligence Radar

A Beginner's Guide to Everything We Built

Programming, APIs, Geospatial Math, Data Pipelines,
Git, and Machine Learning — Explained From Scratch

Built by Kaartik Arora, with Claude

How to use this document

This isn't a project writeup — it's a teaching document, written for someone who has never written a line of code before. Every concept here was learned while building a real project (a Bangalore ride-price prediction tool), and this document walks through those same concepts in the same order, assuming nothing in advance.

Each part builds on the one before it, the same way the project itself was built block by block. Nothing in a later part requires anything except what came before it in this document. Real code from the actual project is included throughout, along with real mistakes that were made and caught along the way — because seeing how a bug actually happens and gets fixed teaches more than only ever seeing correct code.

If a sentence uses a word you don't recognize, check the Glossary at the very end first — it's built to be flipped to at any point, not just read in order.

Part 0 — Before You Start: What Is Any of This?

What is a computer program?

A computer, on its own, does nothing on its own initiative — it only ever follows exact, literal instructions. A **program** (also called **code** or a **script**) is a written list of those instructions, in a language the computer can understand. **Python** is one such language — a set of rules for how to write instructions a computer can follow, chosen for this project because it reads closer to plain English than most programming languages, and has a huge collection of pre-built tools (called **libraries**) for almost anything you'd want to do.

Files, folders, and paths

A Python program is just a text file, saved with a `.py` ending (e.g. `geocoding.py`) so your computer and your editor know it's Python code, not a plain document. A **path** is the exact location of a file on your computer, written as a chain of folders:

`D:\PROJECTS\commute-intelligence-radar\geocoding.py` means "go to drive D, then the PROJECTS folder, then commute-intelligence-radar, then the file named `geocoding.py`." The **project root** or **working directory** is just whichever folder you're currently "in" when running commands — the same way you can only see what's on your current desk, not every desk in the building, until you walk to a different one.

The terminal: typing commands instead of clicking

Most computer use involves clicking icons and buttons. A **terminal** (also called a **command line**, **console**, or **shell**) is a text-only way of controlling your computer — you type a command, press Enter, and it runs immediately. This project used PowerShell and Git Bash (both terminal programs for Windows) throughout, inside a panel at the bottom of VS Code (see below).

A few commands used constantly throughout this project, decoded:

```
cd foldername      # "change directory" -- move into a folder
cd /d D:\PROJECTS  # move into a folder on a DIFFERENT drive letter (Windows quirk)
dir                # list what's in the current folder (Windows)
python file.py      # run a Python file
pip install name    # download and install a Python library
```

Real trap hit early on: on Windows, plain `cd D:\PROJECTS` from a C: drive terminal *looks* like it worked (no error) but silently does nothing — `cd` alone only changes folders within the *current* drive letter. The fix is the `/d` switch: `cd /d D:\PROJECTS`, which also switches drives. Always check the prompt text itself to confirm where you actually ended up, rather than assuming a command worked just because nothing turned red.

VS Code: where the actual writing happens

VS Code (Visual Studio Code) is a free code editor — a text editor built specifically for writing and running code, with helpful extras like color-coding ("syntax highlighting"), error detection, and a built-in terminal panel (opened with Ctrl + the backtick key, usually top-left under Esc) so you don't need a separate window for typing commands.

Running a file, and what "crashing" actually means

Typing `python geocoding.py` in the terminal tells Python: "read this file, top to bottom, and follow every instruction in it, in order." If every instruction is valid, it runs to completion (successfully, silently, or printing whatever the code told it to print). If Python hits an instruction it can't understand or can't complete, it stops immediately and prints an **error message** (sometimes called a **traceback** or a **crash**) explaining what went wrong and on which exact line. This is not a mysterious failure — it's Python being extremely precise about exactly what confused it, which is genuinely useful once you know how to read it (see the error table in Part 1).

pip: installing other people's code

Nobody writes everything from scratch. A **library** (or **package** or **module**) is code someone else already wrote and published, that you can reuse. `pip` is Python's built-in tool for downloading libraries: `pip install requests` downloads and installs the `requests` library so any Python file on your computer can now `import requests` and use it. This project used several real libraries this way: `requests` (talking to websites/APIs), `h3` (Uber's open-source spatial indexing tool), `pandas` (working with tables of data), and `scikit-learn` (machine learning).

Part 1 — Programming Fundamentals

Types: the different kinds of values

Every value in Python has a **type**, and Python enforces the difference strictly. The four types used constantly in this project: **int** (a whole number, like 5), **float** (a decimal number, like 5.3 — almost every real-world measurement in this project, like distance and price, is a float), **str** ("string" — text, always wrapped in quotes, like "Indiranagar"), and **bool** (a value that's only ever True or False). You can't silently mix them — "5" + 3 is an error, because one side is text and the other is a number, even though a human reading it knows exactly what was meant. This strictness is deliberate: it forces you to be explicit, which prevents a whole category of silent, confusing bugs.

Variables and functions

A **variable** is a named box holding a value: `lat = 12.97` stores the number 12.97 under the name `lat`. From that point on in the code, writing `lat` means "whatever value is currently stored there" — you're not retyping the number, you're referring to the box. A **function** is a reusable block of code with a name, defined with `def`, so you can write logic once and use it many times instead of copy-pasting the same instructions everywhere they're needed:

```
def geocode(place_name):  
    # place_name is a *parameter* -- a placeholder for whatever  
    # value gets passed in when the function is called  
    ...  
    return lat, lon
```

Calling `geocode("Indiranagar")` runs the function with `place_name` set to "Indiranagar" for that one run — call it again with a different place name, and the whole function runs fresh with the new value, remembering nothing from the previous call. `return` hands a value back to whoever called the function, so it can be stored in a variable or used immediately: `result = geocode("Indiranagar")` stores whatever `geocode` returned into `result`.

print() vs return — a distinction that trips up almost everyone at first: `print()` just displays something in the terminal, for a human to look at — the value disappears immediately after. `return` hands the value back into the program itself, so other code can actually use it. A function that only `print()`s its answer, instead of returning it, is a dead end — nothing else in the program can access what it "said."

Indentation: why whitespace matters in Python

In many languages, spacing is just cosmetic. In Python, **indentation (leading spaces) is part of the actual grammar** — it's how Python knows which lines belong "inside" a function, an `if`, or a loop, versus which lines are outside it, at the normal level. Two lines with different indentation are treated as being in genuinely different places in the program's structure, even if they sit right next to each other visually.

```
def estimate_price(distance_km, hour):
    base = 40 + 12 * distance_km      # indented -- inside the function
    return base                       # indented -- inside the function

print("done")                        # NOT indented -- outside the function,
                                    # runs regardless of whether estimate_price
was ever called
```

Real bug caused by exactly this: in this project, a few test lines were meant to sit inside an `if __name__ == "__main__":` block (see Part 1 below), but were accidentally left at zero indentation — visually near the block, but not actually inside it. Python ran them unconditionally, every single time the file was imported anywhere, causing silent extra network calls and stray unlabeled output. Indentation isn't a style choice in Python — getting it wrong changes what the program actually does, without necessarily causing any error at all.

Dictionaries and lists

A **list** is an ordered collection: `[1, 2, 3]`. A **dictionary** stores key-value pairs — you look things up by name, not position:

```
VEHICLE_TYPES = {
    "Auto": {"base_fare": 40, "per_km": 20},
    "Mini": {"base_fare": 40, "per_km": 12},
}
VEHICLE_TYPES["Auto"]["base_fare"]  # -> 40
```

Real bug from the project: a variable was defined as `Bangalore_bounds` (capital B) in one file, but imported as `bangalore_bounds` (lowercase) in another. Python is case-sensitive and exact-match — these are two completely different names as far as it's concerned, even though they look almost identical. Result: `ImportError: cannot import name 'Bangalore_bounds'`.

Loops and conditionals

```
for vehicle_type in VEHICLE_TYPES:    # loops over dictionary keys
    print(vehicle_type)

if is_friday and 17 <= hour <= 19:    # 'and' combines conditions
    peak *= 1.15
```

Real bug: `&&` was used instead of Python's `and` — that's the "and" operator in languages like C, Java, and JavaScript, but Python spells it out as the actual word. Result: immediate `SyntaxError`.

f-strings and string formatting

An f-string lets you embed variables directly inside text using curly braces:

```
print(f"{pickup_name} -> {drop_name}: {distance:.1f} km")
# .1f means "format as a decimal with exactly 1 digit after the point"
```

Modules, imports, and file organization

Each `.py` file is a **module**. `from geocoding import geocode` pulls a specific function from another file into the current one. This project deliberately split responsibilities across files: `geocoding.py` (place names to coordinates), `spatial.py` (distance/routing math), `pricing.py` (the price formula) — each file owns one job, and other files reuse it rather than duplicating logic.

The `if __name__ == "__main__":` pattern

```
if __name__ == "__main__":
    print(geocode("Indiranagar, Bangalore"))
```

Code inside this block only runs when the file is executed directly (e.g. `python geocoding.py`), not when another file imports functions from it. Without this guard, test/demo code runs *every single time* the file gets imported anywhere.

Real bug: two test lines were accidentally left outside this block, at the very bottom of `geocoding.py`, unindented. Because they weren't inside `if __name__`, they ran on *every single import* — meaning every other file that used `geocoding.py` silently made two extra network calls and printed stray, unlabeled `True/False` output every time it ran. Fix: move all test code inside one `if __name__` block at the very bottom of the file.

Virtual environments

Normally, `pip install` puts a package somewhere global on your machine, shared across every project. A **virtual environment** (`venv`) is an isolated copy of that package pool, tied to one project folder. When a terminal prompt shows `(.venv)`, every install and every run happens inside that isolated bubble.

Real bug: `pandas` had been installed earlier, in a regular terminal. Later, inside an active `.venv`, Python could no longer find it — not because it wasn't installed anywhere, but because it wasn't installed *in this specific isolated environment*. Fix: reinstall inside the active `venv`.

Common errors, decoded

Error	What it actually means
<code>ModuleNotFoundError</code>	You imported a library that was never pip-installed (or not installed in this environment).
<code>SyntaxError</code>	Invalid Python grammar -- often a typo like <code>&&</code> instead of <code>and</code> , or a missing character.
<code>NameError</code>	You referenced a name (variable/function) that doesn't exist at this point in the code.
<code>ImportError</code>	The exact name you tried to import doesn't exist in that file -- often a capitalization mismatch.
<code>TypeError</code>	An operation got the wrong type/number of arguments (e.g. calling a function with 2 args when it needs 4).
<code>ValueError</code>	Right type, but an invalid value -- e.g. <code>float("two hundred")</code> can't convert text to a number.

[Errno 2] No such file or directory	The exact filename you typed doesn't exist where you're looking -- almost always a typo.
-------------------------------------	--

Part 2 — Working with APIs

What an API request actually is

An API (Application Programming Interface) request is your program asking another computer, over the internet, for information. A **GET request** asks for data (as opposed to a **POST request**, which sends data to be saved). Python's `requests` library makes this simple:

```
import requests

response = requests.get(url, params=params, headers=headers)
data = response.json()  # parses the response text into Python data
```

Query parameters, headers, and JSON

Query parameters (`params`) are extra bits of information tacked onto a request, telling the server what you want. **Headers** carry metadata about the request itself — e.g. Nominatim (the geocoding service used in this project) requires a `User-Agent` header identifying your app, so it knows who's using its free service. **JSON** is a structured, text-based data format — `.json()` converts a raw JSON response into a normal Python list/dictionary you can work with directly.

Rate limits and fair-use APIs

Several free services used in this project — Nominatim (geocoding) and OSRM (road routing) — are free, public, and require no account or API key, but ask that you don't exceed **1 request per second**. This is why a small pause (`time.sleep(1)`) appears after each call to these services in the project's code — it's respecting a shared, free resource, not a technical requirement of the API itself.

Real example: geocoding a place name

```
def geocode(place_name):
    url = "https://nominatim.openstreetmap.org/search"
    params = {"q": place_name, "format": "json"}
    headers = {"User-Agent": "CommuteIntelligenceRadar/1.0 (email@example.com)"}

    response = requests.get(url, params=params, headers=headers)
    data = response.json()

    if not data:
        return None  # Nominatim returns [] if nothing was found

    lat = float(data[0]["lat"])  # values come back as text -- must convert
    lon = float(data[0]["lon"])
    return lat, lon
```

Real bug: `data[1]["lon"]` was used instead of `data[0]["lon"]` — `data` is a list of *different candidate places*, so this silently mixed the latitude of the best-guess result with the longitude of a

completely different second-best guess. It didn't crash, and the numbers looked almost right — which made it dangerous. Lesson: bugs that don't crash but quietly produce slightly-wrong data are the ones worth being most careful about.

Part 3 — Geospatial Basics

Latitude, longitude, and bounding boxes

Every point on Earth has a latitude (north-south position) and longitude (east-west position). A **bounding box** is a rough rectangle defined by min/max latitude and min/max longitude — a cheap way to check "is this point roughly inside this city."

```
bangalore_bounds = {
    "min_lat": 12.83, "max_lat": 13.14,
    "min_lon": 77.45, "max_lon": 77.75,
}

def is_in_blr(lat, lon):
    return (lat <= bangalore_bounds["max_lat"] and lat >= bangalore_bounds["min_lat"]
            and lon <= bangalore_bounds["max_lon"] and lon >= bangalore_bounds["min_
lon"])
```

The haversine formula (straight-line distance)

Distance between two lat/lon points can't be found by simple subtraction — the Earth is a sphere, not a flat grid. The **haversine formula** calculates real straight-line ("as the crow flies") distance between two points on a sphere:

```
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371 # Earth's radius in km
    lat1_rad, lon1_rad = math.radians(lat1), math.radians(lon1)
    lat2_rad, lon2_rad = math.radians(lat2), math.radians(lon2)
    dlat = lat2_rad - lat1_rad
    dlon = lon2_rad - lon1_rad
    a = math.sin(dlat/2)**2 + math.cos(lat1_rad)*math.cos(lat2_rad)*math.sin(dlon/2)
    **2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return R * c
```

Trig functions need input in **radians**, not degrees, so coordinates are converted first. This formula assumes a perfect sphere (Earth is very slightly flattened), giving well under 0.5% error — entirely fine for this use case.

Worked example, step by step: Indiranagar (12.9732913, 77.6443636) to Koramangala (12.9352, 77.6146). Converting all four values to radians, then computing the difference (dlat, dlon), then plugging into the formula for a and c, gives a final distance of **5.32 km**. Running the exact same two points through OSRM (real road routing, covered next) instead gives **7.23 km** — a ratio of about 1.36×. That gap, between straight-line and real road distance, is the entire reason OSRM exists in this project.

H3: hexagonal spatial indexing

H3 is a real, open-source (Apache 2.0) library originally built by Uber, freely usable with no account or API key — it's just a math library, same category as installing NumPy. It divides the map into hexagonal cells at different resolutions (zoom levels). Hexagons are useful here because, unlike squares, every neighboring hexagon is equidistant — no "diagonal is farther than sideways" distortion.

```
import h3
def to_h3(lat, lon, resolution=8):
    return h3.latlng_to_cell(lat, lon, resolution)
```

Straight-line distance vs. real road distance

A critical realization made mid-project: roads aren't straight lines. Haversine gives you "as the crow flies" distance, but actual driving distance is almost always longer — sometimes a lot longer. The ratio of real road distance to straight-line distance is called **circuitry**.

First attempt (wrong): a flat multiplier ($\times 1.4$) was applied to every route to approximate road distance. This was challenged with a real counter-example: a mostly-direct route has circuitry close to 1.0, while a route needing a specific highway detour (Bellandur to Kempegowda Airport) has real circuitry around 1.8 $\times$ (confirmed: ~ 30 km straight-line vs. 45-56km real road distance, from multiple sources). A single constant can't represent both correctly.

The real fix: OSRM (Open Source Routing Machine)

OSRM is a free, open-source routing engine with a public demo server (`router.project-osrm.org`) — no key, no card, same 1 req/sec fair-use policy as Nominatim. It computes the actual real road-network route between two points, replacing the approximated multiplier entirely:

```
def get_road_distance_km(lat1, lon1, lat2, lon2):
    # Note: OSRM wants "lon,lat" order -- opposite of how coordinates
    # are normally spoken, a common trap in geospatial APIs (GeoJSON does this too)
    url = f"https://router.project-osrm.org/route/v1/driving/{lon1},{lat1};{lon2},{lat2}"
    response = requests.get(url, params={"overview": "false"})
    data = response.json()
    if data.get("code") != "Ok":
        return None
    return data["routes"][0]["distance"] / 1000    # meters -> km
```

Part 4 — Pricing Logic & Real-World Calibration

Modeling a smooth curve with `math.exp()`

Instead of an on/off "is it rush hour" switch, a smooth bell curve represents how gradually traffic builds toward and away from a peak. `math.exp()` (Euler's number, $e \approx 2.718$, raised to a power) creates this shape: it equals 1 exactly at the center, and shrinks smoothly toward 0 the further you get from it.

```
def peak_factor(hour):
    morning = math.exp(-((hour - 9) ** 2) / (2 * 1.2 ** 2))
    evening = math.exp(-((hour - 19.5) ** 2) / (2 * 1.5 ** 2)) * 1.27
    return max(morning, evening)
```

The "width" number (1.2, 1.5) controls how gradual the curve is — a bigger width means a gentler, broader peak; a smaller width means a sharper, narrower one.

Worked example, morning peak (center=9, width=1.2):

At hour=9 (exact peak): $(9-9)^2=0$, so the exponent is 0, and `math.exp(0) = 1.0` — full strength.
At hour=7 (2 hours before): $(7-9)^2=4$, divided by $(2 \times 1.2^2)=2.88$, giving an exponent of about -1.39 — `math.exp(-1.39) ≈ 0.25`, a quarter-strength peak, correctly representing traffic still building up.
At hour=3 (dead of night, 6 hours before): $(3-9)^2=36$, exponent ≈ -12.5 , `math.exp(-12.5) ≈ 0.0000037` — essentially zero, correctly representing that there's no rush hour at 3am. This is the entire mechanism: distance from the peak, squared, shrinks the result toward zero the further away you get, smoothly rather than as an on/off switch.

What a function like this can't know: the limits of a hand-written model

A hand-written formula like `peak_factor` only knows what it was explicitly told to look for — commute-hour congestion. When real logged prices were compared against this formula's predictions late at night, every single real price came back higher than predicted, because the formula had no concept of late-night driver scarcity (fewer drivers willing to be out at 1am, a real supply-side effect, completely separate from traffic congestion). This is exactly the kind of gap that motivates using machine learning (Part 8) instead of only ever hand-tuning more constants: a model trained on real data can discover patterns nobody thought to explicitly write a rule for.

The most important habit: sourcing real numbers, not guessing

Early versions of this pricing model used plausible-sounding but invented constants (a 1.8x max surge, a flat night rate). Each one was challenged and replaced with something real and citable:

Constant	Replaced with	Real source
Guessed 1.8x max surge	Legal 0.5x-2.0x band	India's 2025 MoRTH Aggregator Guidelines
Equal morning/evening peaks	Evening 1.27x stronger	TomTom 2023 Bengaluru Traffic Index
Auto = govt. meter rate	Auto = real reported app fare	Meter rate is documented as NOT what apps actually charge

Flat 1.4x circuitry factor	Real OSRM road routing	Circuitry varies wildly by specific route (1.0x-1.8x)
----------------------------	------------------------	---

Just as important: numbers that *couldn't* be sourced (peak center times, the Friday-evening multiplier magnitude) were explicitly labeled as unverified assumptions in the project's README, rather than presented with false confidence alongside the real ones.

Mapping a signal onto a legal range

Rather than adding surge bonuses on top of 1.0x (which could technically exceed what's legal), the final formula computes a 0-to-1 "how busy right now" signal, then maps that directly onto the real legal band:

```
raw_signal = min(peak * weekend_damp + (0.3 if is_rainy else 0), 1.0)
surge = MIN_SURGE + (MAX_SURGE - MIN_SURGE) * raw_signal
# raw_signal=0 -> surge=0.5 (floor). raw_signal=1 -> surge=2.0 (ceiling). Never exceeds either.
```

A full worked example, start to finish

Putting every piece of Part 4 together for one real call: `estimate_price(distance_km=5.3, hour=18.5, is_weekend=False, is_rainy=False, vehicle_type="Mini")` — a 5.3km Mini ride at 6:30pm on a weekday.

```
1. base = 40 + 12 * 5.3 = 103.6           (Mini's base_fare + per_km * distance)
2. peak_factor(18.5) -> ~0.80           (close to the 19.5 evening peak center)
3. weekend_damp = 1.0                     (it's a weekday, no damping)
4. raw_signal = min(0.80 * 1.0 + 0, 1.0) = 0.80
5. surge = 0.5 + (2.0 - 0.5) * 0.80 = 1.70
6. price = 103.6 * 1.70 = 176.12
```

Every number in that chain traces back to something explained earlier in this document — nothing here is hidden or magic, it's six ordinary lines of arithmetic, chained together.

Part 5 — Data Pipelines

Reading and writing CSV files

```
import csv

# Writing
with open("data.csv", "a", newline="") as f:  # "a" = append, "w" = overwrite
    writer = csv.DictWriter(f, fieldnames=[...])
    writer.writerow(row)

# Reading
with open("data.csv") as f:
    rows = list(csv.DictReader(f))  # each row becomes a dictionary
```

Important trap: everything read back from a CSV comes back as *text*, even numbers and True/False values. `row["is_weekend"] == "True"` compares against the literal text "True" — writing `== True` (no quotes) instead would silently always be False, since a string can never equal a real boolean.

Synthetic data: simulating what you don't have

Rather than needing thousands of real observed prices (impractical for a personal project), a **synthetic data generator** creates many simulated scenarios — random routes, random times — and runs each one through the (sourced, calibrated) pricing formula. This produces a large training set instantly, at the cost of only being as accurate as the formula generating it.

Manual logging: capturing real ground truth

A separate, small tool asks the person to type in a real price they observed in the actual app. This is slow to accumulate but far more valuable per data point — it's the only way to check whether the simulated data is actually correct.

.gitignore: what NOT to track

Generated files (like a synthetic dataset, regenerable any time by rerunning a script) and secret files (API keys, credentials) shouldn't be committed to version control. A `.gitignore` file lists filenames Git should never track:

```
# Generated data -- regenerate with data_generator.py, don't track
training_data.csv
real_prices.csv

# Secrets -- never commit these
.env
*.pem
credentials.json
```

Part 6 — Git & GitHub

Git vs. GitHub

Git is a program on your own computer that tracks changes to files over time — it works fully offline.

GitHub is a website that hosts copies of Git projects online, so they're backed up and shareable. "Push" sends your local saved checkpoints up to GitHub.

Core vocabulary

Repository (repo): a project folder Git is tracking.

Commit: a saved checkpoint with a message describing what changed.

Push: upload local commits to GitHub.

Clone: download a copy of a GitHub repo to your computer.

The everyday loop

```
git status                # see what's changed
git add filename.py       # stage a file for commit
git commit -m "describe the change" # save a checkpoint
git push                  # upload to GitHub
```

This loop repeats constantly. Committing in small, real chunks as work happens — not one giant upload at the end — leaves a commit history that shows genuine, believable progress over time, which is itself a signal worth having on a portfolio project.

Part 7 — pandas & Feature Engineering

What pandas actually is

pandas is a library for working with tabular data (rows and columns, like a spreadsheet) in Python. Its core object is the **DataFrame** — a table you can query, filter, and transform with short, purpose-built operations instead of hand-written loops.

```
import pandas as pd
df = pd.read_csv("training_data.csv")      # read entire file as a table
df[["distance_km", "hour"]]               # select specific columns
pd.concat([df1, df2], axis=0)              # stack rows (more rows)
pd.concat([df1, df2], axis=1)              # stack columns side by side (more columns)
```

One-hot encoding: turning categories into numbers

A model only understands numbers. A text column like vehicle type ("Auto"/"Mini"/"Sedan") can't just be numbered 1/2/3 — that would falsely imply an order and a distance between categories that don't actually exist (that Sedan is somehow "3x" Auto). **One-hot encoding** instead creates one 0/1 column per category:

```
vehicle_dummies = pd.get_dummies(df["vehicle_type"], prefix="vt")
# "Mini" becomes: vt_Auto=0, vt_Mini=1, vt_Sedan=0 -- no false ordering implied
```

Sim-to-real blending

A real, standard ML technique: train primarily on a large, cheap synthetic dataset (teaches the general shape of a problem), then weight or fine-tune with a smaller amount of real, expensive-to-get data (corrects the model toward reality). In this project, real logged rows were duplicated several times before being combined with synthetic data, so a small number of real observations meaningfully influenced training instead of being drowned out by a much larger synthetic set.

Part 8 — Machine Learning Basics

X and y

Think of it like a practice exam with an answer key. **X** (capital, by convention) is the *question* — everything known before an outcome happens (distance, hour, vehicle type). **y** (lowercase) is the *correct answer* — the actual price. A model looks at many (X, y) pairs during training and works out the pattern connecting them; afterward, it can guess y for a brand-new X it's never seen.

Train/test split: proving a model actually learned something

If a model is evaluated on the exact same data it trained on, a high score doesn't prove it learned a real pattern — it might just have memorized answers, the same way memorizing exam answers isn't the same as understanding a subject. The fix: hold back a portion of data (commonly 20%) that the model never sees during training, and evaluate only on that.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# random_state=42 makes the random split reproducible -- same split every run
```

What a decision tree actually is

Before gradient boosting makes sense, it helps to see one single decision tree first. A **decision tree** is a series of yes/no questions that narrows down toward a prediction — genuinely similar to a flowchart or a game of 20 Questions. For predicting ride price, a single simple tree might look like: "Is the hour past 17? If yes, is distance over 10km? If yes, predict ~450. If no, predict ~200. If the hour wasn't past 17, is it a weekend? ..." and so on. Each question splits the data into two groups, trying to make each resulting group as "similar" (close in price) as possible. A single tree, on its own, is usually a fairly rough predictor — it only asks a limited number of questions before giving up and guessing.

Gradient boosting: many weak trees, correcting each other

A GradientBoostingRegressor doesn't rely on one tree — it builds many small, individually weak decision trees, one after another, typically 100 or more. The first tree makes a rough guess for every row. The second tree's entire job is looking specifically at *where the first tree was wrong*, and predicting a correction for just that error. The third tree corrects what's still wrong after trees one and two. This continues for however many trees were requested, and the final prediction is all of them added together. Each individual tree is weak and rough on its own, but the combination, all correcting each other's mistakes in sequence, becomes a genuinely strong predictor. This is fundamentally different from the hand-written formula in Part 4: nobody tells it "mornings peak at 9am" — it has to discover that pattern itself, by noticing that rows around hour=9 tend to have higher prices than its current guess, and building a tree that corrects specifically for that.

```
from sklearn.ensemble import GradientBoostingRegressor
model = GradientBoostingRegressor(random_state=42)
model.fit(X_train, y_train) # the actual learning step
predictions = model.predict(X_test) # guess on data it's never seen
```

MAE: Mean Absolute Error

For every test row, take the absolute difference between predicted and actual value, then average across all test rows. An MAE of 0.14 means predictions are, on average, off by about 0.14 in whatever unit is being predicted. Lower is better; the number's scale depends entirely on what's being predicted (a surge multiplier vs. a rupee price will naturally have very different-looking MAE values for similar relative accuracy).

Feature importance: what the model actually relied on

Tree-based models can report how much each input actually mattered to their predictions:

```
importances = pd.Series(model.feature_importances_, index=X.columns)
importances.sort_values(ascending=False)
```

Real finding: a model trained to predict *surge* relied on hour for 88.5% of its decisions, with distance and vehicle type barely mattering at all. This wasn't a bug — the model correctly learned that, in the underlying formula, surge genuinely doesn't depend much on distance or vehicle type. It revealed a design problem, not a modeling problem: surge alone doesn't capture what a rider actually cares about (the final price). The fix was changing what the model predicts — price instead of surge — not adjusting the model itself.

Why a trained model doesn't keep learning on its own

GradientBoostingRegressor is a **batch learner** — it has no built-in way to absorb one new example at a time. New real data has zero effect on an already-trained, saved model until training is explicitly rerun on the full dataset (old + new combined). This is standard practice in real production ML systems ("retrain nightly" is a completely normal pattern), not a limitation unique to a small project.

```
import joblib
joblib.dump({"model": model, "vehicle_columns": vehicle_columns}, "surge_model.pkl")
# saved once, loaded instantly later -- no need to retrain every time the app runs
```

Glossary

API — A way for your program to request information from another computer over the internet.

Bounding box — A rough rectangle defined by min/max latitude and longitude, used to check if a point is roughly inside a region.

Circuitry — The ratio of real road distance to straight-line distance between two points.

Commit — A saved checkpoint in Git, with a message describing what changed.

CSV — Comma-Separated Values -- a simple text format for tabular data.

DataFrame — pandas' core object: an in-memory table of rows and columns.

Dictionary — A Python data structure storing key-value pairs, looked up by name.

Feature importance — A measure of how much each input mattered to a trained model's predictions.

f-string — A Python string that lets you embed variable values directly using {curly braces}.

Geocoding — Converting a place name into latitude/longitude coordinates.

Git — A program that tracks changes to files over time, running locally on your computer.

GitHub — A website that hosts Git repositories online.

Haversine formula — A formula calculating real straight-line distance between two points on a sphere.

H3 — An open-source hexagonal spatial indexing system, originally built by Uber.

JSON — A structured, text-based format commonly used for API responses.

MAE (Mean Absolute Error) — The average absolute difference between a model's predictions and the real values.

Module — A single Python file, reusable by other files via import.

One-hot encoding — Converting a text category into several 0/1 columns, one per category, so a model can use it without implying a false order.

OSRM — Open Source Routing Machine -- a free service computing real road-network routes and distances.

Parameter / argument — A parameter is a function's named placeholder input; an argument is the actual value passed in when calling it.

Query parameter — Extra information attached to an API request specifying what you want.

Rate limit — A cap on how often you're allowed to call an API, often to keep a free/shared service usable for everyone.

Sim-to-real — Training a model mostly on cheap synthetic data, then correcting it with a smaller amount of real observed data.

Train/test split — Holding back a portion of data the model never trains on, to fairly evaluate whether it actually learned a pattern.

Virtual environment (venv) — An isolated set of installed Python packages, tied to one specific project.

X and y — Standard ML notation: X is the input features (the 'question'), y is the target being predicted (the 'answer').